

Deviation Apply – three fixes

Cancel still saves the deviation, the same deviation is inserted twice, and the flight dialog has to be replaced. Every change below is additive and guarded, so no existing behaviour changes for any other preference page.

Problem	Cause	Fixed in
Cancel still saves	saveAircraftConfigurations() returns normally when the user cancels – it never throws – so the deviation save in the try block runs anyway.	Steps 1–3
Deviation saved twice	After Apply the model still has deviationId null, so the next Apply creates it again.	Step 4
Flight dialog	MgsPreferenceStoreManager opens it whenever the aircraft is dirty.	Steps 5–6

Cancel must not save

Step 1 — MgsPreferenceStoreManager – new field

com.eurocopter.mgs.presentation.configuration.store.MgsPreferenceStoreManager

Add beside the other private static fields, just after FLIGHT_SYNTHESIS_DIRTY_AIRCRAFTS. A new field read only by our code changes nothing for anyone else.

```
// BCR-6605 true when the last saveAircraftConfigurations() was confirmed by the user
private static boolean lastSaveConfirmed = false;

public static synchronized boolean wasLastSaveConfirmed() {
    return lastSaveConfirmed;
}
```

Step 2 — MgsPreferenceStoreManager → saveAircraftConfigurations – reset the flag

First line of the method, before try {.

```
public static synchronized void saveAircraftConfigurations() {
    lastSaveConfirmed = false;           // <<< add this line
    try {
        // Save first the preferences that are not historicized: ...
    }
```

Step 3 — MgsPreferenceStoreManager → saveAircraftConfigurations – set it on OK

Inside the existing if (mgsPreferenceDialogHandler.getOkCancel(...)), as its first line. The flag is only written here, so Cancel leaves it false.

```
if (mgsPreferenceDialogHandler.getOkCancel(configurationHistorizationDto)) {
    lastSaveConfirmed = true;           // <<< add this line
    Flight applicableFlight = mgsPreferenceDialogHandler.getApplicableFlight(
        configurationHistorizationDto);
    ...
}
```

Also handle the case where nothing was dirty. Wrap the existing if (!dirtyAircraftsConfigurations.isEmpty()) block with an else, so a save with no dirty aircraft still counts as confirmed:

```
if (dirtyAircraftsConfigurations.isEmpty()) {
    lastSaveConfirmed = true;           // nothing to confirm, nothing cancelled
} else {
    MgsPreferenceDialogHandler mgsPreferenceDialogHandler = new MgsPreferenceDialogHandler();
    ...                               // the existing block, unchanged
}
```

These three edits only add a field and two assignments. No existing line is removed or reordered, and no other caller reads the flag, so every other preference page behaves exactly as before.

Step 4 — HealthAlarmsComposite → performApply

mgs-health-app/.../healthalarms/HealthAlarmsComposite.java

Replace the whole method. saveDeviations now runs only when the save was confirmed, and before clearAircraftConfigurations so the store is still populated.

```

public void performApply() {
    healthAlarmsPreferenceStore.innerSave();
    try {
        MgsPreferenceStoreManager.saveAircraftConfigurations();
        if (MgsPreferenceStoreManager.wasLastSaveConfirmed()) { // <<< BCR-6605
            healthAlarmsPreferenceStore.saveDeviations(); // <<< BCR-6605
        }
        MgsPreferenceStoreManager.clearAircraftConfigurations();
    } catch (Exception e) {
        LOGGER.error("Error while saving health preferences", e);
    }
}
}

```

No duplicate rows

Step 5 — HealthAlarmsPreferenceStore → saveDeviations

mgs-health-app/.../healthalarms/store/HealthAlarmsPreferenceStore.java

Replace the whole method. Two changes: collect the models that were sent, and mark them afterwards so a second Apply skips them. The three LOGGER.info debug lines go.

```

public void saveDeviations() {
    List<IndicatorDeviationDto> newDeviations = new ArrayList<>();
    List<WCThresholdModel> savedModels = new ArrayList<>(); // <<< BCR-6605
    Date now = new Date();

    for (Entry<Aircraft, WCMechanicalComponentModel> entry : data.entrySet()) {
        Multimap<Indicator, WCThresholdModel> thresholds = getComponentThresholds(entry.getValue());
        for (WCThresholdModel model : thresholds.values()) {
            if (!model.isDeviated() || model.getDeviationId() != null) {
                continue;
            }
            newDeviations.add(toDeviationDto(model, now));
            savedModels.add(model); // <<< BCR-6605
        }
    }

    if (!newDeviations.isEmpty()) {
        try {
            healthBusiness.createDeviations(newDeviations);
            // BCR-6605 mark as saved so the next Apply does not insert the same row again
            for (WCThresholdModel model : savedModels) {
                model.setDeviationId("SAVED");
            }
        } catch (Exception e) {
            LOGGER.error("Could not save deviations", e);
        }
    }
}
}

```

"SAVED" is a placeholder. The proper fix is to have createDeviations return the generated ids and assign the real one to each model – worth doing before the Deviation List tab, which will need those ids to delete a row. Mark it with a TODO for now.

The confirmation popup

Step 6 — CoreMessages.properties – and every language file

mgs-client-common/src/com/eurocopter/mgs/core/presentation/i18n/CoreMessages.properties

Two new keys. Place them alphabetically near the other deviation keys.

```

deviationConfirmTitle=Save deviation
deviationConfirmMessage=Do you want to save the deviation?

```

Step 7 — HealthAlarmsComposite → performApply – ask first

First lines of the method, before healthAlarmsPreferenceStore.innerSave(). Returning early leaves everything untouched.

```

public void performApply() {
    // BCR-6605 confirm before saving
    if (!AlertDialog.openConfirm(getShell(),
        Messages.CORE.getString("deviationConfirmTitle"),
        Messages.CORE.getString("deviationConfirmMessage"))) {
        return;
    }

    healthAlarmsPreferenceStore.innerSave();
    ...
}

```

Imports: org.eclipse.jface.dialogs.MessageDialog and com.eurocopter.mgs.core.presentation.i18n.Messages.

This adds the new popup but does not remove the flight dialog. The flight dialog opens because the aircraft is marked dirty, and it belongs to MgsPreferenceStoreManager, which every preference page shares – suppressing it there would change behaviour for Data Collection, Flight Synthesis, FDCR and the rest. The only safe way to stop it for a deviation-only edit is to not mark the aircraft dirty in the first place, which means WCTreeCellModifier.modify skipping MgsPreferenceStoreManager.markAsDirty(...) when the edited column is a deviation field. That is a design decision: it also means a deviation no longer creates a configuration version at all. Confirm with the senior before implementing it.

Order and test

#	Action
1	Steps 1–3 in MgsPreferenceStoreManager, then Step 4 in HealthAlarmsComposite
2	Step 5 in HealthAlarmsPreferenceStore
3	Steps 6–7 for the popup
4	Rebuild the client only – no server change in this set, no redeploy
5	Edit a Value, type a TE, Apply, press Cancel on the flight dialog – expect no new row in indicator_deviation
6	Repeat and press OK – expect exactly one new row
7	Press Apply again without changing anything – expect no second row
8	Open another preference page, change something, Apply – expect the flight dialog to behave exactly as before

Item 8 is the regression check that matters. The three edits to MgsPreferenceStoreManager are additive, but that class is shared, so prove it on a page that has nothing to do with this BCR.